FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Andreu Liam C. Gamboa
Section:	AN43
Professor:	Crisola G. Tan
Date:	10/12/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

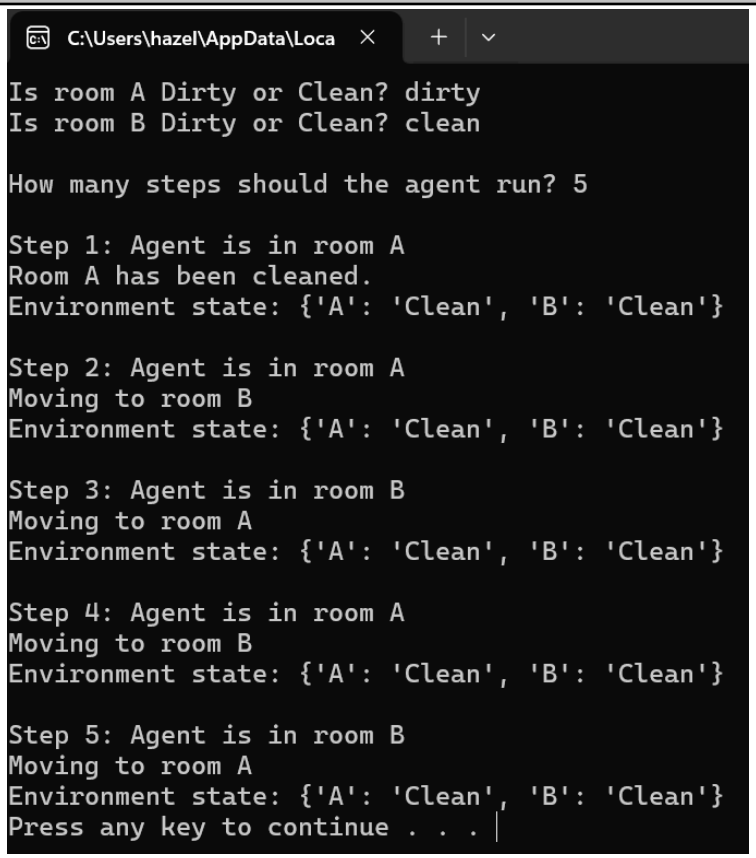
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

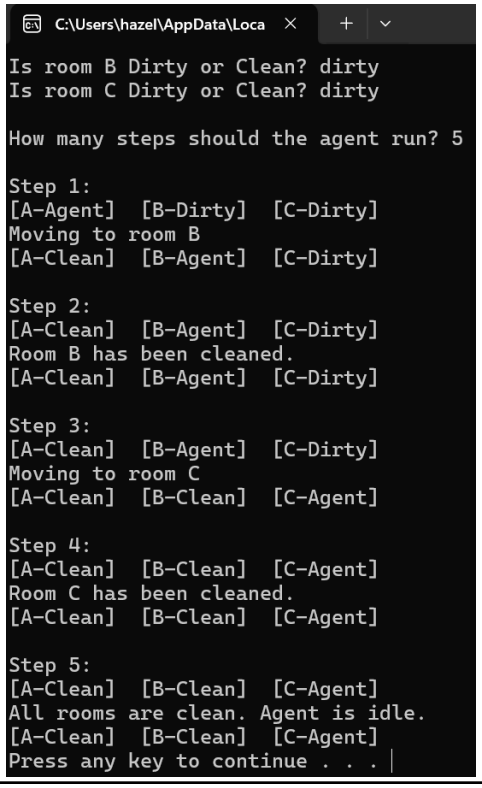
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

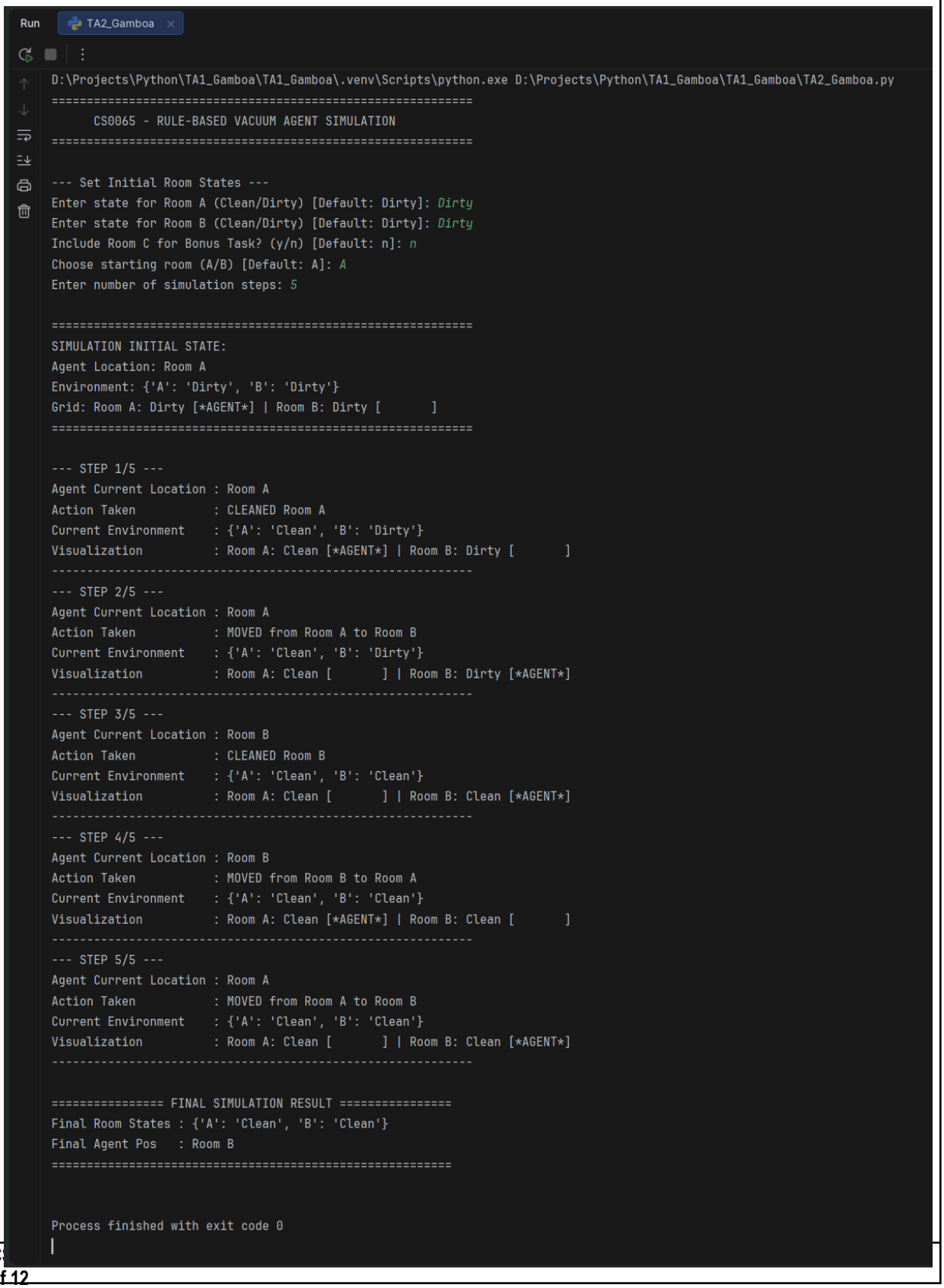
Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

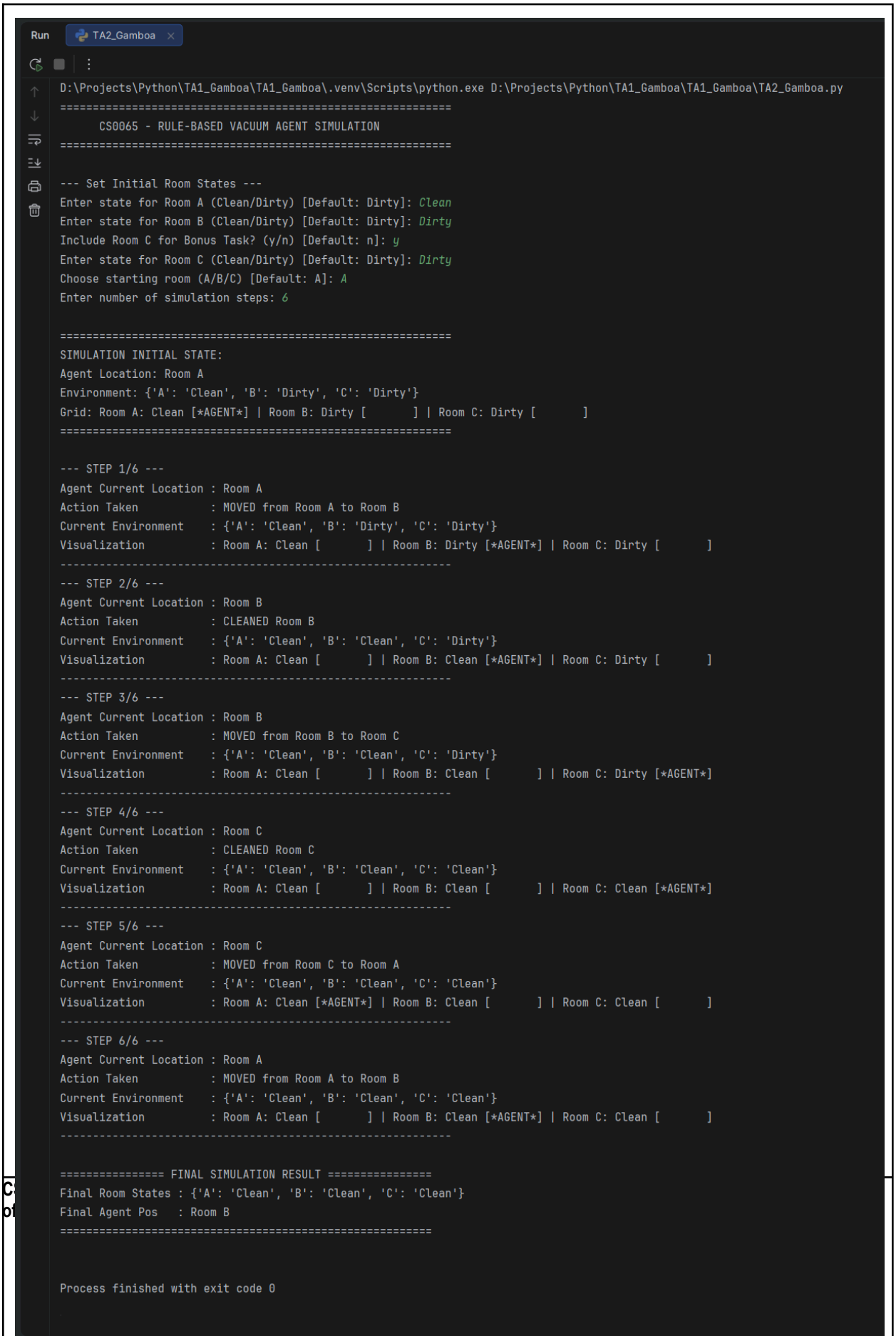
Sample Output (Bonus Task)



2-Room


Run
TA2_Gamboa
D:\Projects\Python\TA1_Gamboa\TA1_Gamboa\.venv\Scripts\python.exe D:\Projects\Python\TA1_Gamboa\TA1_Gamboa\TA2_Gamboa.py
CS0065 - RULE-BASED VACUUM AGENT SIMULATION
--- Set Initial Room States ---
Enter state for Room A (Clean/Dirty) [Default: Dirty]: Dirty
Enter state for Room B (Clean/Dirty) [Default: Dirty]: Dirty
Include Room C for Bonus Task? (y/n) [Default: n]: n
Choose starting room (A/B) [Default: A]: A
Enter number of simulation steps: 5
SIMULATION INITIAL STATE:
Agent Location: Room A
Environment: {'A': 'Dirty', 'B': 'Dirty'}
Grid: Room A: Dirty [*AGENT*] | Room B: Dirty [ ]
--- STEP 1/5 ---
Agent Current Location : Room A
Action Taken : CLEANED Room A
Current Environment : {'A': 'Clean', 'B': 'Dirty'}
Visualization : Room A: Clean [*AGENT*] | Room B: Dirty [ ]
--- STEP 2/5 ---
Agent Current Location : Room A
Action Taken : MOVED from Room A to Room B
Current Environment : {'A': 'Clean', 'B': 'Dirty'}
Visualization : Room A: Clean [ ] | Room B: Dirty [*AGENT*]
--- STEP 3/5 ---
Agent Current Location : Room B
Action Taken : CLEANED Room B
Current Environment : {'A': 'Clean', 'B': 'Clean'}
Visualization : Room A: Clean [ ] | Room B: Clean [*AGENT*]
--- STEP 4/5 ---
Agent Current Location : Room B
Action Taken : MOVED from Room B to Room A
Current Environment : {'A': 'Clean', 'B': 'Clean'}
Visualization : Room A: Clean [*AGENT*] | Room B: Clean [ ]
--- STEP 5/5 ---
Agent Current Location : Room A
Action Taken : MOVED from Room A to Room B
Current Environment : {'A': 'Clean', 'B': 'Clean'}
Visualization : Room A: Clean [ ] | Room B: Clean [*AGENT*]
FINAL SIMULATION RESULT
Final Room States : {'A': 'Clean', 'B': 'Clean'}
Final Agent Pos : Room B
Process finished with exit code 0


3-Room



```
Run  TA2_Gamboa

D:\Projects\Python\TA1_Gamboa\TA1_Gamboa\.venv\Scripts\python.exe D:\Projects\Python\TA1_Gamboa\TA1_Gamboa\TA2_Gamboa.py
============================================================
      CS0065 - RULE-BASED VACUUM AGENT SIMULATION
============================================================

--- Set Initial Room States ---
Enter state for Room A (Clean/Dirty) [Default: Dirty]: Clean
Enter state for Room B (Clean/Dirty) [Default: Dirty]: Dirty
Include Room C for Bonus Task? (y/n) [Default: n]: y
Enter state for Room C (Clean/Dirty) [Default: Dirty]: Dirty
Choose starting room (A/B/C) [Default: A]: A
Enter number of simulation steps: 6

============================================================
SIMULATION INITIAL STATE:
Agent Location: Room A
Environment: {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}
Grid: Room A: Clean [*AGENT*] | Room B: Dirty [       ] | Room C: Dirty [       ]
============================================================

--- STEP 1/6 ---
Agent Current Location : Room A
Action Taken           : MOVED from Room A to Room B
Current Environment    : {'A': 'Clean', 'B': 'Dirty', 'C': 'Dirty'}
Visualization          : Room A: Clean [       ] | Room B: Dirty [*AGENT*] | Room C: Dirty [       ]
------------------------------------------------------------
--- STEP 2/6 ---
Agent Current Location : Room B
Action Taken           : CLEANED Room B
Current Environment    : {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}
Visualization          : Room A: Clean [       ] | Room B: Clean [*AGENT*] | Room C: Dirty [       ]
------------------------------------------------------------
--- STEP 3/6 ---
Agent Current Location : Room B
Action Taken           : MOVED from Room B to Room C
Current Environment    : {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}
Visualization          : Room A: Clean [       ] | Room B: Clean [       ] | Room C: Dirty [*AGENT*]
------------------------------------------------------------
--- STEP 4/6 ---
Agent Current Location : Room C
Action Taken           : CLEANED Room C
Current Environment    : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
Visualization          : Room A: Clean [       ] | Room B: Clean [       ] | Room C: Clean [*AGENT*]
------------------------------------------------------------
--- STEP 5/6 ---
Agent Current Location : Room C
Action Taken           : MOVED from Room C to Room A
Current Environment    : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
Visualization          : Room A: Clean [*AGENT*] | Room B: Clean [       ] | Room C: Clean [       ]
------------------------------------------------------------
--- STEP 6/6 ---
Agent Current Location : Room A
Action Taken           : MOVED from Room A to Room B
Current Environment    : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
Visualization          : Room A: Clean [       ] | Room B: Clean [*AGENT*] | Room C: Clean [       ]
------------------------------------------------------------

================ FINAL SIMULATION RESULT ================
Final Room States : {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
Final Agent Pos   : Room B
============================================================


Process finished with exit code 0
```

Your Code - Laboratory Tasks

Source Code

```
"""
CS0065 - Intelligent Systems
Technical Assessment 2: Rule-Based Agent (Vacuum Cleaner)
File: TA2_Gamboa.py
"""


# ================= Task 1: Environment Setup =================
class Environment:
    def __init__(self, rooms=None):
        # Default initialization for rooms (Task 1 & Bonus Task)
        if rooms is None:
            self.rooms = {"A": "Dirty", "B": "Dirty"}
        else:
            self.rooms = rooms

    def is_dirty(self, room):
        """Check if the given room is dirty."""
        return self.rooms.get(room, "").lower() == "dirty"

    def clean_room(self, room):
        """Clean the specified room."""
        if room in self.rooms:
            self.rooms[room] = "Clean"

    def display_grid(self, agent_location):
        """Bonus Task: Text-based grid visualization."""
        room_visuals = []
        for room in sorted(self.rooms.keys()):
            status = self.rooms[room]
            marker = "[*AGENT*]" if room == agent_location else "[       ]"
            room_visuals.append(f"Room {room}: {status:<5} {marker}")
        return " | ".join(room_visuals)


# ================= Task 2: Rule-Based Agent =================
class RuleBasedVacuumAgent:
```

```
    def __init__(self, starting_room="A"):
        self.location = starting_room

    def perceive_and_act(self, env):
        """
        Rule 1: If the current room is dirty, clean it.
        Rule 2: If the current room is clean, move to another room.
        """
        if env.is_dirty(self.location):
            # Rule 1
            env.clean_room(self.location)
            return f"CLEANED Room {self.location}"
        else:
            # Rule 2
            old_location = self.location
            self.move(env)
            return f"MOVED from Room {old_location} to Room {self.location}"

    def move(self, env):
        """Switch rooms in cyclic order (supports 2 or more rooms)."""
        available_rooms = sorted(list(env.rooms.keys()))
        current_index = available_rooms.index(self.location)
        next_index = (current_index + 1) % len(available_rooms)
        self.location = available_rooms[next_index]


# ================= Task 3: Simulation Loop =================
def run_simulation():
    print("=" * 60)
    print("      CS0065 - RULE-BASED VACUUM AGENT SIMULATION")
    print("=" * 60)

    # Task 1: Allow user to set initial environment state
    print("\n--- Set Initial Room States ---")
    state_a = input("Enter state for Room A (Clean/Dirty) [Default: Dirty]: ").strip().capitalize()
    if state_a not in ["Clean", "Dirty"]:
        state_a = "Dirty"

    state_b = input("Enter state for Room B (Clean/Dirty) [Default: Dirty]: ").strip().capitalize()
    if state_b not in ["Clean", "Dirty"]:
        state_b = "Dirty"

    # Optional Bonus Room C prompt
```

```
    include_c = input("Include Room C for Bonus Task? (y/n) [Default: n]: ").strip().lower()
    initial_rooms = {"A": state_a, "B": state_b}

    if include_c == 'y':
        state_c = input("Enter state for Room C (Clean/Dirty) [Default: Dirty]: ").strip().capitalize()
        if state_c not in ["Clean", "Dirty"]:
            state_c = "Dirty"
        initial_rooms["C"] = state_c

    start_loc = input(f"Choose starting room ({'/'.join(sorted(initial_rooms.keys()))}) [Default: A]:
").strip().upper()
    if start_loc not in initial_rooms:
        start_loc = "A"

    steps = int(input("Enter number of simulation steps: "))

    # Initialize environment and agent
    env = Environment(initial_rooms)
    agent = RuleBasedVacuumAgent(starting_room=start_loc)

    print("\n" + "=" * 60)
    print("SIMULATION INITIAL STATE:")
    print(f"Agent Location: Room {agent.location}")
    print(f"Environment: {env.rooms}")
    print(f"Grid: {env.display_grid(agent.location)}")
    print("=" * 60 + "\n")

    # Step loop
    for step in range(1, steps + 1):
        print(f"--- STEP {step}/{steps} ---")
        current_loc = agent.location
        action = agent.perceive_and_act(env)

        print(f"Agent Current Location : Room {current_loc}")
        print(f"Action Taken           : {action}")
        print(f"Current Environment    : {env.rooms}")
        print(f"Visualization          : {env.display_grid(agent.location)}")
        print("-" * 60)

    print("\n================ FINAL SIMULATION RESULT ================")
    print(f"Final Room States : {env.rooms}")
    print(f"Final Agent Pos   : Room {agent.location}")
    print("==========================================================\n")
```

```
if __name__ == "__main__":
    run_simulation()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1
The agent follows two condition-action (if-then) rules: ● Rule 1 (Cleaning): If the current room status perceived by the agent is Dirty, execute the clean_room action on the current room. ● Rule 2 (Movement): If the current room status perceived by the agent is Clean, execute the move action to shift position to the adjacent room.

2
The decision is made through the agent's percept cycle via is_dirty(self.location). The agent accesses the environment dictionary to read the state of its current tile. If the boolean check returns True, cleaning takes priority; if it returns False (meaning the tile is already clean), it triggers room switching.

3

If both rooms are initialized to Clean, Rule 1 is never triggered. At every simulation step, the agent perceives its current room as clean and executes Rule 2, causing it to continuously cycle/alternate between Room A and Room B without performing any cleaning operations.

4

To support N rooms (e.g., A, B, C, ...):

- Store room identifiers in an ordered list or graph topology and compute transitions using modular arithmetic: next_index = (current_index + 1) % len(rooms).
- Alternatively, implement an internal memory/search policy (such as Breadth-First Search or priority targeting) where the agent queries which rooms remain Dirty and navigates toward the closest uncleaned room instead of relying on a hardcoded toggle.